We can simplify our specs even further using the subject method from rspec-core instead of repeating the recipe-creation code. This construct is the equivalent of calling let(:subject) { ... }:

10-exploring-rspec-expectations/06/spec/cookie_recipe_subject_spec.rb
```
RSpec.describe CookieRecipe, '#ingredients' do
  subject { CookieRecipe.new.ingredients }
  it { is_expected.to include(:butter, :milk, :eggs) }
  it { is_expected.not_to include(:fish_oil) }
end
```

The subject method defines how to build the object we're testing. RSpec gives us is_expected as shorthand for expect(subject). The phrase it is_expected reads nicely in each spec here, though in larger projects we tend to favor more explicit let constructs for clarity's sake.

Once again, we get the same documentation output:

```
$ rspec spec/cookie_recipe_subject_spec.rb --format documentation

CookieRecipe#ingredients
  should include :butter, :milk, and :eggs
  should not include :fish_oil

Finished in 0.00225 seconds (files took 0.08616 seconds to load)
2 examples, 0 failures
```

Here's an alternate form of this one-liner syntax that looks even more like the generated descriptions:

10-exploring-rspec-expectations/06/spec/cookie_recipe_should_spec.rb
```
RSpec.describe CookieRecipe, '#ingredients' do
  subject { CookieRecipe.new.ingredients }
  it { should include(:butter, :milk, :eggs) }
  it { should_not include(:fish_oil) }
end
```

Wait, didn't we just say should was problematic in *What Happened to should?*, on page 171? The problem with the old should from RSpec 2 and before isn't the name; it's the fact that RSpec had to monkey-patch the core Object class to implement it.

This should is different: it's just a local alias for expect(subject).to. You can use either should or is_expected.to, whichever you prefer.

We recommend you use this one-liner syntax sparingly. It's possible to over-emphasize brevity and rely too much on one-liners. To keep our enthusiasm in check, let's go over a few of the downsides of generated descriptions: